Relatório — Streaming de Áudio com GStreamer (RTP/UDP)

Nov 11, 2026 · @Lucas

Disciplina: Sistemas Multimídia (CMP1103/C01), PUC Goiás — 2026/2 **Professor:** Prof. Esp. Thalles Bruno Santos **Grupo 7:** Leonardo Campos, Matheus Henrique, Wisley Victor, Lucas Rodrigues

1. Objetivo

O grupo desenvolveu uma aplicação em C++ com GStreamer que transmite áudio em tempo real por RTP/UDP, com dois programas: um **transmissor** e um **receptor**. No teste realizado, o receptor recebeu cerca de 50 pacotes por segundo com **0% de perda** e jitter médio abaixo de 0,1 ms.

A atividade pedia:

- transmissor e receptor em C/C++, com host e porta configuráveis;
- controle de jitter com o `rtpjitterbuffer` e latência configurável;
- diagnóstico de perdas de pacotes;
- diagrama das pipelines e registro dos testes.

2. Arquitetura

O áudio sai do transmissor comprimido em Opus, dentro de pacotes RTP, e atravessa a rede por UDP. O receptor faz o caminho inverso e usa o `rtpjitterbuffer` para reordenar os pacotes e absorver as variações de atraso.

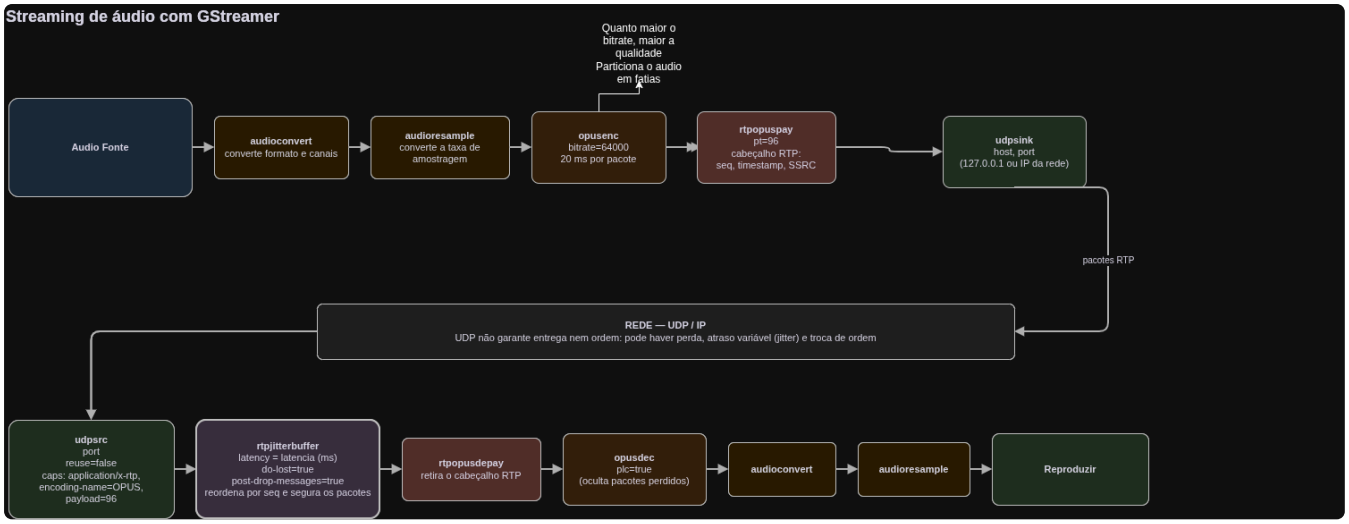
Transmissor:

`fonte → audioconvert → audioresample → opusenc → rtpopuspay → [netsim] → udpsink`

Receptor:

`udpsrc → rtpjitterbuffer → rtpopusdepay → opusdec → audioconvert → audioresample → autoaudiosink`

As pipelines foram montadas elemento por elemento no código, seguindo os Basic Tutorials 2 e 6 do GStreamer: cada elemento é criado com `gst_element_factory_make`, configurado com `g_object_set`, adicionado com `gst_bin_add_many` e ligado com `gst_element_link_many`. O diagrama abaixo mostra as duas pipelines e onde atua cada diagnóstico.



3. Implementação

3.1 Transmissor (`transmissor.cpp`)

Elemento	Configuração	Função
`audiotestsrc` / `autoaudiosrc`	`is-live=TRUE`, `freq=440`	Gera um tom de 440 Hz em tempo real, ou captura o microfone
`audioconvert` + `audioresample`	—	Adaptam formato e taxa de amostragem ao que o codificador aceita (negociação de caps)
`opusenc`	`bitrate=64000`	Comprime em Opus, em quadros de 20 ms (50 pacotes/s)
`rtpopuspay`	`pt=96`	Encapsula cada quadro em RTP, com número de sequência e timestamp
`netsim` (opcional)	`drop-probability`, `max-delay`	Simula perda e jitter para os testes
`udpsink`	`host`, `port`	Envia os pacotes por UDP ao receptor

3.2 Receptor (`receptor.cpp`)

Elemento	Configuração	Função
`udpsrc`	`port` , caps RTP/Opus (48 kHz, pt 96), `reuse=FALSE`	Recebe os pacotes; os caps informam o formato dos dados
`rtpjitterbuffer`	`latency` , `do-lost=TRUE` , `post-drop-messages=TRUE`	Segura os pacotes pelo tempo da latência, reordena e detecta perdas
`rtpopusdepay`	—	Retira o cabeçalho RTP
`opusdec`	`plc=TRUE`	Decodifica e oculta pacotes perdidos (*Packet Loss Concealment*)
`audioconvert` + `audioresample`	—	Adaptam o áudio à placa de som
`autoaudiosink`	—	Reproduz no dispositivo padrão (ou `fakesink` com o argumento `nula`)

3.3 Parâmetros de linha de comando

Programa	Uso	Padrão
Transmissor	`./transmissor [host] [porta] [teste|mic] [perda %] [jitter ms]`	127.0.0.1, 5000, teste, 0, 0
Receptor	`./receptor [porta] [latencia ms] [nula]`	5000, 200 ms

Os dois programas verificam se cada elemento foi criado e se a ligação e a mudança para PLAYING funcionaram. Os erros em tempo de execução chegam pelo bus e são impressos com o nome do elemento que falhou.

4. Diagnóstico de perdas

O receptor usa três recursos do próprio `rtpjitterbuffer` para mostrar o que acontece com os pacotes:

Recurso	Como é lido	Saída no terminal
Evento de perda (`do-lost`)	*Pad probe* na saída do jitterbuffer, que captura o evento `GstRTPPacketLost`	`[PERDA] pacote N perdido`
Pacote descartado (`post-drop-messages`)	Mensagem `drop-msg` lida no bus	`[DESCARTE] pacote N chegou depois do prazo (too-late)`
Estatísticas (`stats`)	Lidas a cada 1 s por um temporizador (`g_timeout_add_seconds`)	`[t=Ns] recebidos, perdidos, atrasados, duplicados, jitter, perda %`

O evento de perda continua até o `opusdec` , que o usa para sintetizar o trecho que faltou. A taxa de perda é calculada como perdidos / (recebidos + perdidos), e o jitter médio vem do campo `avg-jitter` , convertido de nanossegundos para milissegundos.

5. Ambiente e procedimento de teste

O teste foi feito em um PC com Linux, no terminal do VS Code, com transmissor e receptor na mesma máquina (loopback 127.0.0.1). Os programas foram compilados com `make` (g++ com C++17 e `pkg-config gstreamer-1.0`).

1. Compilação:

```
make
```

2. Terminal 1, receptor na porta 5000 com latência de 200 ms:

```
./build/receptor 5000 200
```

3. Terminal 2, transmissor enviando o tom de 440 Hz para a mesma porta, sem simulação de rede:

```
./build/transmissor 127.0.0.1 5000
```

6. Resultados

Em 8 segundos, o receptor recebeu **379 pacotes, sem nenhuma perda, atraso ou duplicação**, a uma taxa média de 50,7 pacotes/s. O tom de 440 Hz foi reproduzido sem falhas.

Saída do transmissor:

```
Transmitindo tom de 440 Hz para 127.0.0.1:5000 (Opus/RTP/UDP) | perda simulada 0.0% | jitter 0-0 ms
```

Estatísticas impressas pelo receptor (latência de 200 ms):

Tempo	Recebidos (acum.)	Pacotes no segundo	Perdidos	Atrasados	Duplicados	Jitter médio (ms)	Perda
1 s	24	24	0	0	0	0,02	0,00%
2 s	78	54	0	0	0	0,04	0,00%
3 s	127	49	0	0	0	0,02	0,00%
4 s	178	51	0	0	0	0,06	0,00%
5 s	227	49	0	0	0	0,03	0,00%
6 s	279	52	0	0	0	0,03	0,00%
7 s	328	49	0	0	0	0,04	0,00%
8 s	379	51	0	0	0	0,07	0,00%

Análise:

- **Taxa de pacotes:** a partir do 2º segundo, chegaram de 49 a 54 pacotes por segundo, o esperado para quadros Opus de 20 ms (1 s / 20 ms = 50). O 1º segundo teve só 24 porque o transmissor foi iniciado depois do receptor, no meio desse intervalo. As pequenas variações (49, 51, 54) vêm do momento em que o temporizador de 1 s lê as estatísticas, não de perdas.
- **Perda:** 0 pacotes perdidos, 0 atrasados e 0 duplicados. Sem simulação de rede, todos os pacotes chegaram dentro do prazo do jitterbuffer.
- **Jitter:** entre 0,02 e 0,07 ms (média de cerca de 0,04 ms), muito abaixo da latência de 200 ms. Isso é esperado no loopback, onde não há roteadores nem disputa de banda.

7. Conclusão

A aplicação cumpriu o objetivo: o áudio foi codificado em Opus, encapsulado em RTP, enviado por UDP para uma porta configurada e reproduzido no receptor sem perdas. O Opus reduziu o fluxo de 768 kbps (PCM 48 kHz mono) para 64 kbps, e o RTP deu a cada pacote o número de sequência e o timestamp de que o receptor precisa para reordenar e detectar perdas.

O `rtpjitterbuffer` com latência de 200 ms absorveu com folga o jitter medido (menos de 0,1 ms). Os diagnósticos funcionaram, mostrando as estatísticas a cada segundo; como não houve perdas neste teste, não apareceram linhas `[PERDA]` nem `[DESCARTE]`.

A limitação é que o loopback representa uma rede ideal. Numa rede real, com jitter maior, a latência passa a ser um compromisso: valores baixos dão menos atraso, mas fazem pacotes atrasados virarem perdas; valores altos evitam perdas, mas atrasam a reprodução. O transmissor já permite simular esse cenário com o `netsim` (argumentos de perda e jitter).

Referências

- GStreamer. Basic tutorial 2: GStreamer concepts.
- GStreamer. Basic tutorial 6: Media formats and Pad Capabilities.
- GStreamer. rtpjitterbuffer.
- GStreamer. opusenc e opusdec.
- IETF. RFC 3550: RTP — A Transport Protocol for Real-Time Applications.
- IETF. RFC 7587: RTP Payload Format for the Opus Speech and Audio Codec.